

PRACTICAL WEEK -1

Session-1

Task1: Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that:

1. Accept a Linux command as input.
2. Create a child process using fork().
3. Execute the command in the child process using an appropriate exec() system call.
4. Allow the parent process to wait for the child using wait().
5. Display the Process ID (PID) of both parent and child processes.

```
GNU nano 8.7.1 command_executor.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    char command[100];

    // Accept Linux command from the user
    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);

    // Remove newline character
    command[strcspn(command, "\n")] = '\0';

    // Create child process
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    // Child process
    if (pid == 0)
    {
        printf("Child Process PID: %d\n", getpid());
        printf("Parent Process PID: %d\n", getppid());

        // Execute the Linux command
        execlp(command, command, (char *)NULL);

        // Executes only if execlp fails
        perror("exec failed");
        exit(1);
    }

    // Parent process
    else
    {
        printf("Parent Process PID: %d\n", getpid());
        printf("Child Process PID: %d\n", pid);

        // Wait for child process to finish
        wait(NULL);

        printf("Child process completed.\n");
    }

    return 0;
}
```

```

shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/practical/week1/task1$ pwd
/home/shamikahasini/OSSP/practical/week1/task1
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/practical/week1/task1$ ls
command_executor.c
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/practical/week1/task1$ gcc command_executor.c -o command_executor
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/practical/week1/task1$ ls
command_executor  command_executor.c
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/practical/week1/task1$ ./command_executor
Enter a Linux command: ls
Parent Process PID: 659
Child Process PID: 660
Child Process PID: 660
Parent Process PID: 659
command_executor  command_executor.c
Child process completed.
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/practical/week1/task1$

```

Observation: The program successfully accepted a Linux command as input and created a child process using `fork()`. The child process executed the entered `ls` command using `exec()`, while the parent process waited for the child process using `wait()`. The Parent PID and Child PID were displayed successfully, and the child process completed execution successfully.

Task2: Using Linux terminal commands (`uname`, `lscpu`, `lsblk`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

```

shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/practical/week1$ cd task2
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/practical/week1/task2$ pwd
/home/shamikahasini/OSSP/practical/week1/task2
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/practical/week1/task2$ uname -a
Linux LAPTOP-D2M8NKKBH 6.8.0-39-microsoft-standard-WSL2 #1 SMP "PREEMPT_DYNAMIC Thu Jun 18 2
12:01:21 UTC 2026 x86_64 GNU/Linux
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/practical/week1/task2$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 2
On-line CPU(s) list:   0-1
Vendor ID:              GenuineIntel
Model name:             12th Gen Intel(R) Core(TM) i5-12450HX
CPU family:             6
Model:                 151
Thread(s) per core:    2
Core(s) per socket:    6
Socket(s):              1
Stepping:               2
BogoMIPS:               5376.00
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov
pat pse36 clflush mmx fxsr sse sse2 ss ht syscall nx rdtscp lm con-
stant_tsc rep_good nopl xtopology tsc_reliable nonstop_tsc pni pcl-
mulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 sdbg fma cx16 xtpr
pdcm pcid sse4_1 sse4_2 movbe popcnt aes xsave avx f16c rdrand hyperv
lahf_lm abm 3dnowprefetch cpuid epb invpcid_single pti tpr_shadow
vmx flexpriority ept vpl dtherm ida arat pln pts hwp hwp_notify hwp_
act_window hwp_epp hwp_pkg_req hfi vmx avx512f avx512dq rdseed adx
smap clflushopt clwb sha_ni xsaveopt xsavec xgetbv1 xsaves umip pku
ospke waitpkg gfni vaes vpclmulqdq rdpid movdiri movdir64b ferm ad-
clear serialize pconfig flush_lld arch_capabilities

Virtualization features:
Virtualization:         VT-x
Hypervisor vendor:      Microsoft
Virtualization type:    full
Caches (sum of all):
  L1:                    288 KiB (6 instances)
  L2:                    192 KiB (6 instances)
  L3:                    7.5 MiB (6 instances)
  L4:                    12 MiB (1 instance)
NUMA:
NUMA node(s):           1
NUMA node0 CPU(s):     0-1
Vulnerabilities:
  Gather data sampling:  Not affected
  Ghostwrite:           Not affected
  Indirect target selection: Not affected
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/practical/week1/task2$

```

```

Ghostwrite: Not affected
Indirect target selection: Not affected
Itlb multihit: Not affected
L1tf: Not affected
Mds: Not affected
Meltdown: Not affected
Mmio stale data: Not affected
Old microcode: Not affected
Reg file data sampling: Mitigation; Clear Register File
Retbleed: Mitigation; Enhanced IBRS
Spec rstack overflow: Not affected
Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2: Mitigation; Enhanced / Automatic IBRS; IBPB conditional; ~IBRS SW sequence; BHI BHI_DIS_S
Srbds: Not affected
Tsa: Not affected
Tsx async abort: Not affected
Vmscape: Not affected

```

```
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/practical/week1/task2$ lsblk
```

```

NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda 8:0 0 356.9M 1 disk
sdb 8:16 0 159.4M 1 disk
sdc 8:32 0 2G 0 disk [SWAP]
sdd 8:48 0 1T 0 disk /mnt/wslg/distro
└─ /

```

```
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/practical/week1/task2$ ps
```

```

PID TTY TIME CMD
330 pts/0 00:00:00 bash
790 pts/0 00:00:00 ps

```

```
shamikahasini@LAPTOP-D2M8NKKBH:~/OSSP/practical/week1/task2$ top
```

```

top - 16:05:05 up 22 min, 1 user, load average: 0.03, 0.02, 0.00
Tasks: 24 total, 1 running, 23 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 5778.6 total, 5165.6 free, 528.3 used, 228.2 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 5250.3 avail Mem

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	24124	15396	11608	S	0.0	0.3	0:00.68	systemd
2	root	20	0	3180	2204	2072	S	0.0	0.0	0:00.00	init-systemd(ub
6	root	20	0	3180	2048	1940	S	0.0	0.0	0:00.00	init
46	root	19	-1	50400	16796	15520	S	0.0	0.3	0:00.18	systemd-journal
78	systemd+	20	0	22416	14436	11940	S	0.0	0.2	0:00.06	systemd-resolve
84	root	20	0	35348	12304	9236	S	0.0	0.2	0:00.38	systemd-udevd
121	root	20	0	2888	1952	1820	S	0.0	0.0	0:00.05	chronyd-starter
143	root	20	0	4472	3148	2900	S	0.0	0.1	0:00.01	cron
148	message+	20	0	8816	5448	4772	S	0.0	0.1	0:00.06	dbus-daemon
157	root	20	0	44096	29808	14668	S	0.0	0.5	0:00.23	networkd-dispat
164	root	20	0	18440	9380	8132	S	0.0	0.2	0:00.07	systemd-logind
195	syslog	20	0	220548	5952	4648	S	0.0	0.1	0:00.08	rsyslogd
226	root	20	0	123460	32764	18156	S	0.0	0.6	0:00.18	unattended-upgr
233	root	20	0	5492	2744	2556	S	0.0	0.0	0:00.01	agetty
256	_chrony	20	0	23592	11072	7128	S	0.0	0.2	0:00.08	chronyd
271	_chrony	20	0	12096	2336	1696	S	0.0	0.0	0:00.00	chronyd
328	root	20	0	3184	1108	980	S	0.0	0.0	0:00.00	SessionLeader
329	root	20	0	3200	1248	1108	S	0.0	0.0	0:00.05	Relay(330)
330	shamikahasini+	20	0	6268	5536	3848	S	0.0	0.1	0:00.12	bash
331	root	20	0	8648	5524	4680	S	0.0	0.1	0:00.01	login
377	shamikahasini+	20	0	22336	13636	11008	S	0.0	0.2	0:00.12	systemd
379	shamikahasini+	20	0	22912	4088	2088	S	0.0	0.1	0:00.00	(sd-pam)
410	shamikahasini+	20	0	6136	5520	3928	S	0.0	0.1	0:00.03	bash
804	shamikahasini+	20	0	8188	6148	4024	R	0.0	0.1	0:00.01	top


```

## Relationship Between Hardware Resources and Operating System Services

The operating system provides an abstraction layer between application software and computer hardware>

### CPU Management|

The operating system manages the CPU by scheduling processes and allocating CPU time to different tas>

### Memory Management

The operating system manages RAM by allocating memory to processes and releasing it when it is no lon>

### Storage Management

The operating system manages storage devices through the filesystem. The 'lsblk' command displays blo>

### Process Management

The operating system creates, schedules, and terminates processes. The 'ps' command provides informat>

### Device and I/O Management

The operating system manages input/output devices through device drivers and system interfaces. This >

## Summary of Observed Results

| Resource | Command | Observation |
|---|---|---|
| System/Kernel | 'uname -a' | Linux kernel and x86_64 WSL2 system information |
| CPU | 'lscpu' | 12 CPUs, 6 cores, 2 threads per core |
| Storage | 'lsblk' | 1 TB disk and 2 GB swap device observed |
| Processes | 'ps' | Running processes and their PIDs displayed |
| Process Monitoring | 'top' | Real-time CPU, memory, and process information displayed |
| Memory | 'free -h' | 5.6 GiB total memory and 2.0 GiB swap observed |

## Key Findings

1. Linux successfully detected and reported the available CPU resources.
2. Storage devices and swap space were identified using 'lsblk'.
3. Running processes were identified using their Process IDs.
4. The 'top' command provided real-time information about system activity.
5. Memory and swap utilization were observed using 'free -h'.
6. The operating system manages hardware resources and provides controlled access to applications.
7. Linux commands provide useful interfaces for monitoring system resources.

## Final Result

The hardware resources and operating system services were successfully investigated using Linux comma>

```

Observation: The Linux terminal commands successfully provided information about the system's hardware resources and operating system services. The uname command displayed kernel and system information, while lscpu showed CPU architecture, 12 CPUs, 6 cores, and 2 threads per core. The lsblk command displayed the available storage devices and swap space. The ps and top commands showed running processes and their resource usage. The OS abstracts CPU, memory, storage, and I/O devices by managing them through system services and providing applications with controlled interfaces instead of requiring direct access to the hardware.